Contents

1 Visual EDA **1**
1.1 The Role of Visuals in EDA . . . 1
1.2 Plot Your Data . . . 1
1.3 The Grammar of Graphics . . . 3

1 Visual EDA

- Recommended reading: @WeissgerberBarLineGraphs2015; @WickhamLayeredGrammarGraphics2010
- Further reading: @HealyDataVisualizationPractical2018, ch. 1; @MatejkaSameStatsDifferent

1.1 The Role of Visuals in EDA

- @TukeyExploratoryDataAnalysis1977 emphasized visual methods for EDA
- Including not just graphs but also structured tables, such as stem-and-leaf displays
- In terms of our 3 models of EDA (ch. **??**)
  - Risk management: With the right plot, our visual system can very efficiently detect aspects of the data that don't show up easily with summary statistics but will break models
    - Outliers
    - Degeneracies (eg, perfect correlation)
    - Skew or bimodal distributions
    - Non-linear relationships
  - Epicycle of analysis:
    - Check expectations about the distribution of variables
    - Develop expectations about relationships between variables
  - Phenomena development:
    - *[something]*

1.2 Plot Your Data

- Summary statistics almost always focus only on central tendency (mean, median) and dispersion (standard deviation, IQR)
  - This is all you would need if the world were made of normal distributions
  - The world is not made of normal distributions [@LyonWhyAreNormal2014]
- We'll illustrate this using The Datasaurus Dozen, a collection of 13 datasets
  - Each dataset has two variables, `x` and `y`

```r
library(tidyverse)
```

```
## -- Attaching packages --------------------------------------- tidyverse 1.3.0 --

## v ggplot2 3.3.2     v purrr   0.3.4
## v tibble  3.0.3     v dplyr   1.0.2
## v tidyr   1.1.2     v stringr 1.4.0
## v readr   1.3.1     v forcats 0.5.0

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
```

```r
library(datasauRus)
dataf = datasaurus_dozen

dataf
```

```
## # A tibble: 1,846 x 3
##    dataset     x     y
##    <chr>   <dbl> <dbl>
##  1 dino     55.4  97.2
##  2 dino     51.5  96.0
##  3 dino     46.2  94.5
##  4 dino     42.8  91.4
##  5 dino     40.8  88.3
##  6 dino     38.7  84.9
##  7 dino     35.6  79.9
##  8 dino     33.1  77.6
##  9 dino     29.0  74.5
## 10 dino     26.2  71.4
## # ... with 1,836 more rows
```

```r
count(dataf, dataset)
```

```
## # A tibble: 13 x 2
##    dataset        n
##    <chr>      <int>
##  1 away         142
##  2 bullseye     142
##  3 circle       142
##  4 dino         142
##  5 dots         142
##  6 h_lines      142
##  7 high_lines   142
##  8 slant_down   142
##  9 slant_up     142
## 10 star         142
## 11 v_lines      142
## 12 wide_lines   142
## 13 x_shape      142
```

- The datasets have the same means, standard deviations, and (Pearson) correlation coefficient
  - p-value of the correlation coefficient is not statistically significant

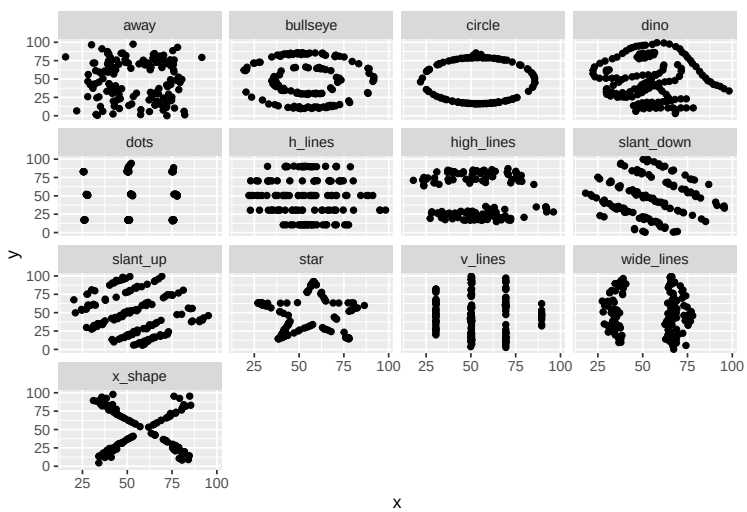
```r
dataf %>%
    group_by(dataset) %>%
    ## This is a more complex summarize() call than we've seen before
    ## 1. Number of rows
    ## 2. "Summarize across the columns x and y, using the functions mean and sd"
    ## 3. Correlation between x and y
    ## 4. p-value for t-test of the null that the correlation = 0
    summarize(n = n(),
              across(.cols = c(x, y),
                     .fns = lst(mean, sd)),
              cor_xy = cor(x, y),
              p_value = cor.test(x, y)$p.value) %>%
    ungroup()
```

```
## `summarise()` ungrouping output (override with `.groups` argument)
```

```
## # A tibble: 13 x 8
##    dataset        n x_mean  x_sd y_mean  y_sd  cor_xy p_value
##    <chr>      <int>  <dbl> <dbl>  <dbl> <dbl>   <dbl>   <dbl>
```

```
##  1 away         142   54.3  16.8   47.8  26.9 -0.0641   0.448
##  2 bullseye     142   54.3  16.8   47.8  26.9 -0.0686   0.417
##  3 circle       142   54.3  16.8   47.8  26.9 -0.0683   0.419
##  4 dino         142   54.3  16.8   47.8  26.9 -0.0645   0.446
##  5 dots         142   54.3  16.8   47.8  26.9 -0.0603   0.476
##  6 h_lines      142   54.3  16.8   47.8  26.9 -0.0617   0.466
##  7 high_lines   142   54.3  16.8   47.8  26.9 -0.0685   0.418
##  8 slant_down   142   54.3  16.8   47.8  26.9 -0.0690   0.415
##  9 slant_up     142   54.3  16.8   47.8  26.9 -0.0686   0.417
## 10 star         142   54.3  16.8   47.8  26.9 -0.0630   0.457
## 11 v_lines      142   54.3  16.8   47.8  26.9 -0.0694   0.412
## 12 wide_lines   142   54.3  16.8   47.8  26.9 -0.0666   0.431
## 13 x_shape      142   54.3  16.8   47.8  26.9 -0.0656   0.438
```

```r
ggplot(dataf, aes(x, y)) +
    geom_point() +
    facet_wrap(vars(dataset))
```



1.3 The Grammar of Graphics

- @WilkinsonGrammarGraphics1999; @WickhamLayeredGrammarGraphics2010
- *Graphs are maps*
  - A graph maps variables in the data to features of geometric objects
  - An individual map is called an *aesthetic*
  - So a graph is a collection of aesthetics applied to a dataset
- **Graphs are models**
  - Simplifications of the data
  - Designed to fit with our cognitive capabilities
  - In ways that support pattern detection, inference
  - Recall Leonelli's relational view of data
  - (Cog sci folks: would you use the term "cognitive affordance" here?)

Figure 1: The Grammar of Graphics regards graphs as mappings from variables in the data to features of geometric objects. This means that every graph is a model.